

IoT-Driven Automated Exam Supervision and Anti-Cheating System

Contents

1)Introduction	3
2) Problem Description.....	3
3) Literature Review	3
4) Project Objectives	4
5) Hardware Architecture & Components	4
6) Software Functionality & Modules.....	4
7) Implementation Timeline	5
8) References	5

1) Introduction

Maintaining academic integrity during examinations is a cornerstone of any educational institution. The rapid miniaturization of communication technology has introduced new challenges, with the emergence of undetectable micro-devices such as nano-earpieces and hidden GSM cards making traditional exam invigilation increasingly difficult.

This project proposes a comprehensive, automated supervision system that merges Internet of Things (IoT) hardware telemetry with continuous Computer Vision analysis. By continuously monitoring the classroom environment for unauthorized radio frequency (RF) transmissions, acoustic anomalies, and suspicious physical behavior throughout the entire exam period, this system acts as a real-time, smart assistant for human proctors.

2) Problem Description

Current anti-cheating measures rely heavily on visual supervision by human proctors or manual searches using handheld metal detectors. These methods suffer from critical flaws:

- **Human Limitations:** A proctor cannot physically monitor 30–50 students simultaneously. Micro-cheating behaviors are practically invisible.
- **Bypassing Traditional Security:** Modern cheating devices use minimal metallic components and are easily concealed in clothing, bypassing standard door checks.
- **Reactive vs. Proactive:** Current detection is reactive. Furthermore, purely manual observation fails to track subtle, repetitive suspicious movements over time.

There is an urgent need for a proactive system that continuously analyzes both the physical and electromagnetic environment of the examination hall to detect the usage of unauthorized communication devices and irregular student behavior.

3) Literature Review

Existing solutions to combat electronic cheating generally fall into distinct categories, each with notable limitations:

1. **Signal Jammers:** While effective at blocking cellular signals, jammers are legally restricted in many regions and disrupt legitimate emergency communications.
2. **Standalone Vision Systems:** Automated proctoring systems relying entirely on cameras often fail to detect students who stare normally at their papers while secretly receiving audio answers via a nano-earpiece.
3. **Handheld RF Detectors:** Security personnel using handheld scanners cause disruption and distraction during the exam.

The Proposed System: This project introduces a fused architecture. It combines continuous video surveillance for behavioral analysis with localized edge nodes for RF and acoustic anomaly detection, providing a multi-layered, foolproof supervision ecosystem.

4) Project Objectives

1. **Continuous Behavioral Analysis:** Implement a robust Computer Vision module using OpenCV to continuously track head poses and detect unauthorized objects in real-time throughout the exam.
2. **Broadband RF Detection:** Design a passive hardware sensor capable of detecting unauthorized wireless transmissions (GSM, 3G, 4G, and Bluetooth).
3. **Acoustic Monitoring:** Implement sound detection thresholds to identify unnatural whispering or verbal communication.
4. **Real-Time Proctor Dashboard:** Develop a scalable backend and an interactive dashboard to aggregate video feeds and sensor telemetry, delivering instant, localized alerts.

5) Hardware Architecture & Components

The physical layer of the project acts as the sensory nervous system, deployed across the examination hall:

- **Main Microcontroller (ESP32):** Serves as the central IoT edge node for sensory data. It continuously gathers RF and acoustic data and transmits it to the backend.
- **Passive RF Peak Detector:** A custom-built wideband receiver utilizing high-speed Schottky diodes to convert ambient high-frequency RF signals into measurable DC voltage.
- **Sound Sensor Module (LM393 or MAX4466):** Calibrated to monitor the ambient noise floor and trigger signals upon detecting localized acoustic peaks.
- **Vision Node (ESP32-CAM / IP Camera):** Positioned to cover specific seating zones. Unlike event-driven models, this node continuously streams video data via the local network to the backend server for uninterrupted frame-by-frame processing.

6) Software Functionality & Modules

To handle continuous video streaming alongside real-time IoT telemetry, the software architecture utilizes Clean Architecture principles, ensuring scalability and separation of concerns.

- **Core Backend (ASP.NET Core Web API):** Acts as the central nervous system. It ingests continuous data streams and routes them efficiently. The backend logic evaluates combined risk scores (matching a spike in RF data with a suspicious head movement).
- **Computer Vision Engine (OpenCV):** A dedicated continuous background service that processes the incoming video streams frame-by-frame. It applies algorithms to track gaze/head orientation and detect predefined forbidden objects.
- **Real-Time Communication (SignalR):** Maintains persistent WebSocket connections between the server and the frontend. When the vision engine or telemetry nodes flag an incident, the SignalR hub pushes a live alert to the proctor's dashboard instantly.

- **Data Persistence (SQL Server & Entity Framework Core):** All exam session configurations, classroom layouts, and logged incident histories (timestamps, desk IDs, and violation snapshots) are securely stored. EF Core manages complex relational mapping between students, sessions, and recorded violations.
- **Frontend Dashboard:** An interactive web interface displaying the live video feed overlay, real-time telemetry graphs, and an instant alert log for the proctor.

7) Implementation Timeline

The project development is structured over a 24-week period to ensure comprehensive design, integration, and testing:

- **Weeks 1–4: Hardware Prototyping & Vision Basics**
Assemble the passive RF detector, calibrate sound sensors, and establish a stable continuous video streaming protocol from the camera node.
- **Weeks 5–8: Edge Firmware & OpenCV Integration**
Program the ESP32 for accurate telemetry reading. Develop the initial Python/C++ OpenCV scripts for head pose tracking and object detection on the continuous stream.
- **Weeks 9–12: Backend Infrastructure**
Set up the ASP.NET Core solution using Clean Architecture. Design the SQL Server database schema and configure Entity Framework Core for data persistence.
- **Weeks 13–16: Real-Time Integration & Dashboarding**
Develop the proctor frontend interface. Integrate SignalR to establish WebSocket connections for live data streaming and instant alert mapping.
- **Weeks 17–20: System Unification & Algorithmic Fusion**
Connect the end-to-end flow. Fuse the OpenCV analytical data with the IoT telemetry data in the backend to generate unified incident reports.
- **Weeks 21–24: Testing, Calibration & Documentation**
Conduct mock exam trials. Optimize the network for continuous video bandwidth, fine-tune sensitivity thresholds, and finalize the graduation thesis.

8) References

[1] "AI-Assisted Devices in University Written Examinations: A Structural Threat to the Degree as a Warrant of Competence," *Preprints.org*, Jun. 2026. [Online]. Available: <https://www.preprints.org/manuscript/202606.2014>